AI Sales Analyst Copilot

Beginner-Friendly Project Documentation

Project type	AI-powered sales analytics and business insight platform
Main goal	Upload a CSV/Excel dataset and get reliable profiling, KPIs, root-cause analysis, Q&A;, charts, and reports.
Prepared for	Beginner explanation, manager review, GitHub documentation, and future development.
Prepared by	Kedar V.
Core design rule	Pandas/NumPy calculate numbers. AI/template explains only verified results.

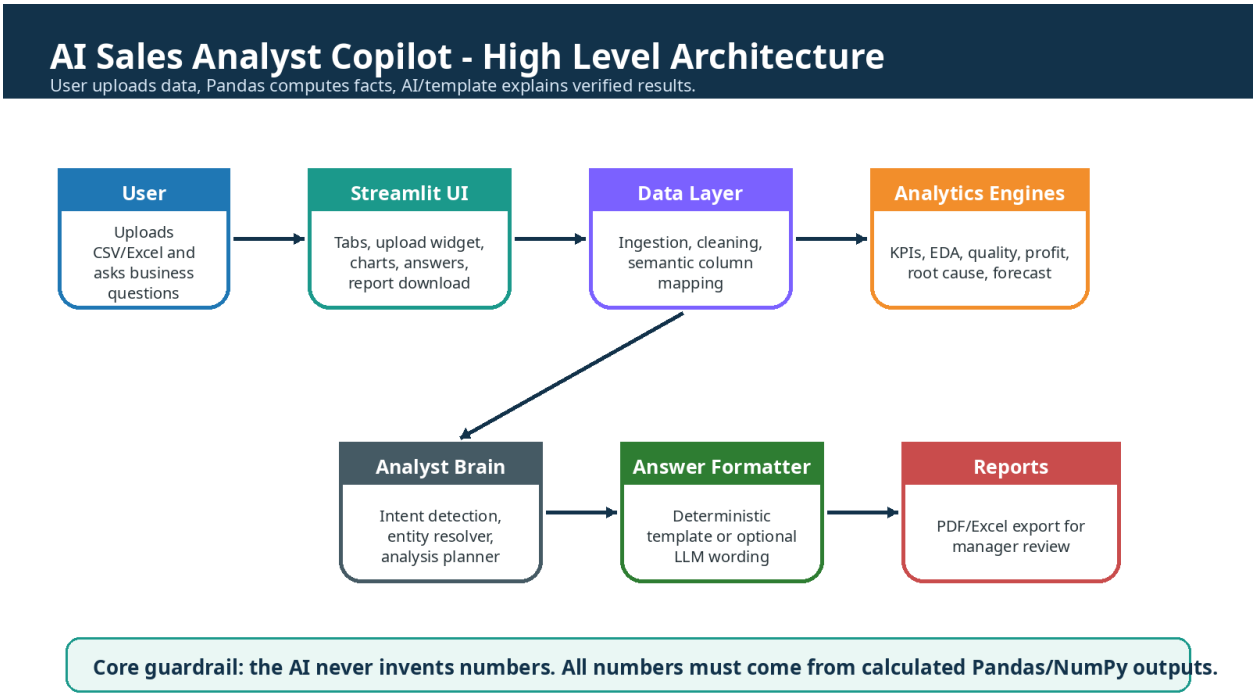


Table of Contents

1. What This Project Is

AI Sales Analyst Copilot is a Streamlit-based analytics application. A user uploads a CSV or Excel sales dataset, and the app automatically explains the dataset, checks data quality, calculates business KPIs, performs exploratory data analysis, identifies profit and loss drivers, answers natural language questions, and generates reports.

The project is not just a dashboard. A dashboard usually shows fixed charts. This project is designed to behave like a junior/senior analyst assistant: it reads the available columns, understands what business role each column plays, calculates results using Python, and then explains the result in simple language.

The most important idea

The LLM must not guess numbers. All numeric answers should come from deterministic code such as Pandas groupby, aggregation, filtering, correlation, date parsing, and statistical calculations. The AI layer is used only for wording, explanation, and business recommendations based on the computed evidence.

User asks	What the app should do
How much sales did we make?	Find the sales/revenue column and calculate the total.
Why is Furniture losing money?	Filter/category-group the dataset, calculate profit by sub-category/product/discount/region, then explain the strongest drivers.
Are there duplicates?	Run duplicate row/column checks and report exact counts.
What should we fix?	Use calculated loss/profit drivers to recommend business actions.

2. Beginner Glossary

Term	Simple meaning	Example in this project
Dataset	A table of information.	A Superstore sales CSV with orders, sales, profit, discount, region, category.
Row	One record in the table.	One order line.
Column	One field/attribute in the table.	Sales, Profit, Order Date, Category.
KPI	A key number used to understand business performance.	Total Sales, Total Profit, Profit Margin, Average Order Value.
EDA	Exploratory Data Analysis: inspecting patterns before making decisions.	Charts, summaries, correlations, top/bottom products.
Data quality	Checking whether the data has technical problems.	Missing values, duplicates, invalid dates, empty columns.
Semantic layer	A mapping between actual column names and business roles.	Revenue, Total Amount, and Sales can all mean sales.
Root cause	The reason behind a result.	Tables may cause furniture losses because of high discount and negative margin.
LLM	A language model used to explain text.	Optional OpenAI/Ollama answer polishing, not number generation.
Deterministic	Same input gives same output because code calculates it.	Pandas sum of Profit column.

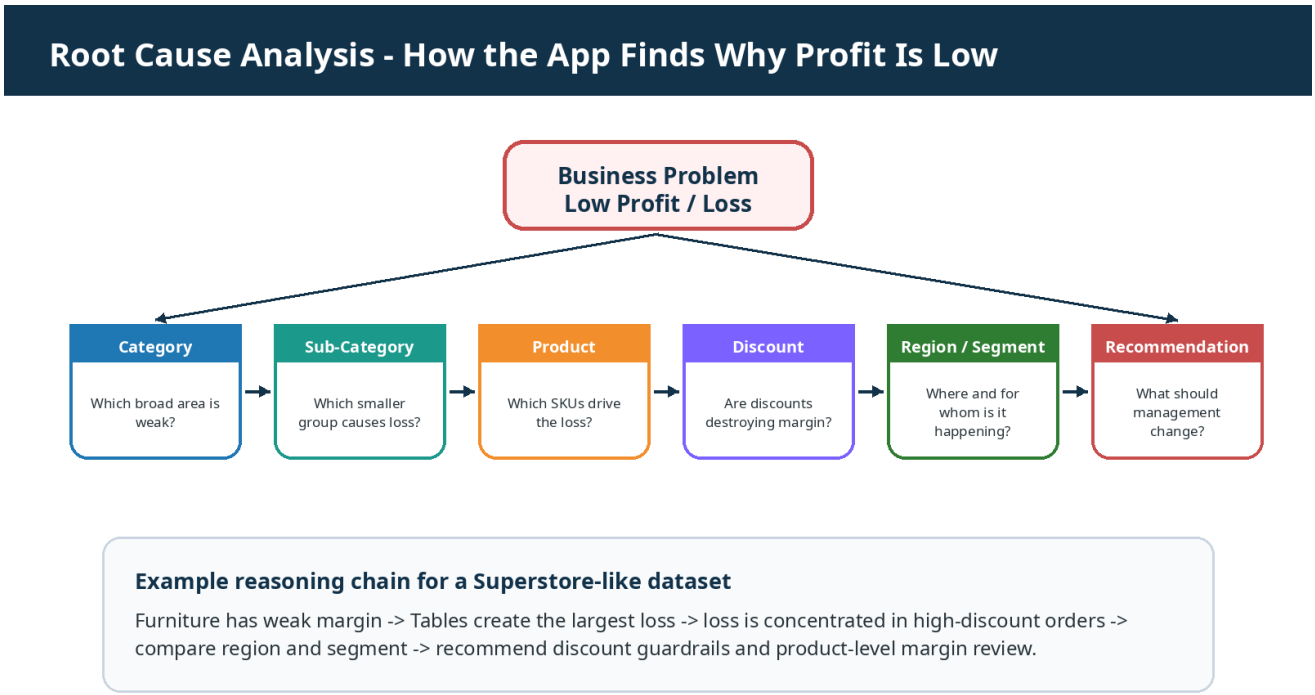
3. Problem Statement and Real-World Example

Business managers often have raw CSV/Excel files but do not know Python, SQL, or Power BI. They need answers such as what is profitable, what is losing money, whether discounts are damaging margins, and which products or regions need attention. A normal chatbot can give confident but wrong answers if it is not grounded in actual calculations.

This project solves that by building a calculated analytics layer first, then adding a natural-language layer above it. The user does not need to write code. The system handles upload, profiling, column detection, calculations, charts, explanations, and export.

Example scenario

A manager uploads a sales dataset and asks: 'What is killing our profit?' The app should not reply vaguely. It should calculate profit by category, sub-category, product, discount band, region, and segment. Then it should answer with evidence, for example: the biggest loss is in Tables, high-discount orders are driving negative profit, and management should review discount policy and supplier/product margin.



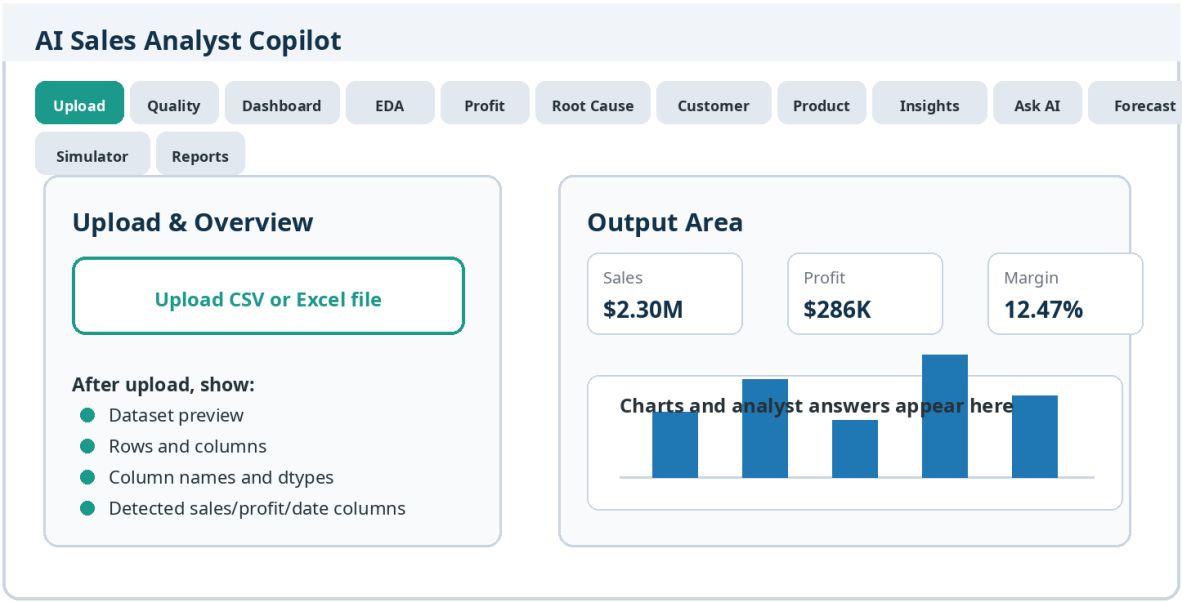
Example reasoning chain for a Superstore-like dataset

Furniture has weak margin -> Tables create the largest loss -> loss is concentrated in high-discount orders -> compare region and segment -> recommend discount guardrails and product-level margin review.

4. What the App Does From Start to Finish

Step	What happens	Why it matters
1. Upload	User uploads CSV, XLSX, or XLS file.	The project starts from real user data, not a hardcoded dataset.
2. Ingestion	The app reads the file into a Pandas DataFrame and handles bad file errors.	Prevents crashes and creates a structured table.
3. Profiling	Rows, columns, dtypes, numeric/categorical/date columns, missing values, duplicate counts.	Gives immediate understanding of the data.
4. Semantic mapping	Detects roles like Sales, Profit, Date, Category, Segment, Customer, Product.	Lets the app work with many different column names.
5. Analysis engines	KPIs, data quality, EDA, profit intelligence, root cause, forecasts, anomalies.	Produces verified calculations.
6. Analyst brain	Routes user questions to the correct module and tool.	Messy questions still reach the right calculation.
7.Answer formatting	Shows result in plain English, with evidence and recommendations.	Makes output useful for non-technical users.
8. Reporting	Exports PDF/Excel reports.	Makes the analysis shareable with a manager.

Streamlit App Layout - What the User Sees



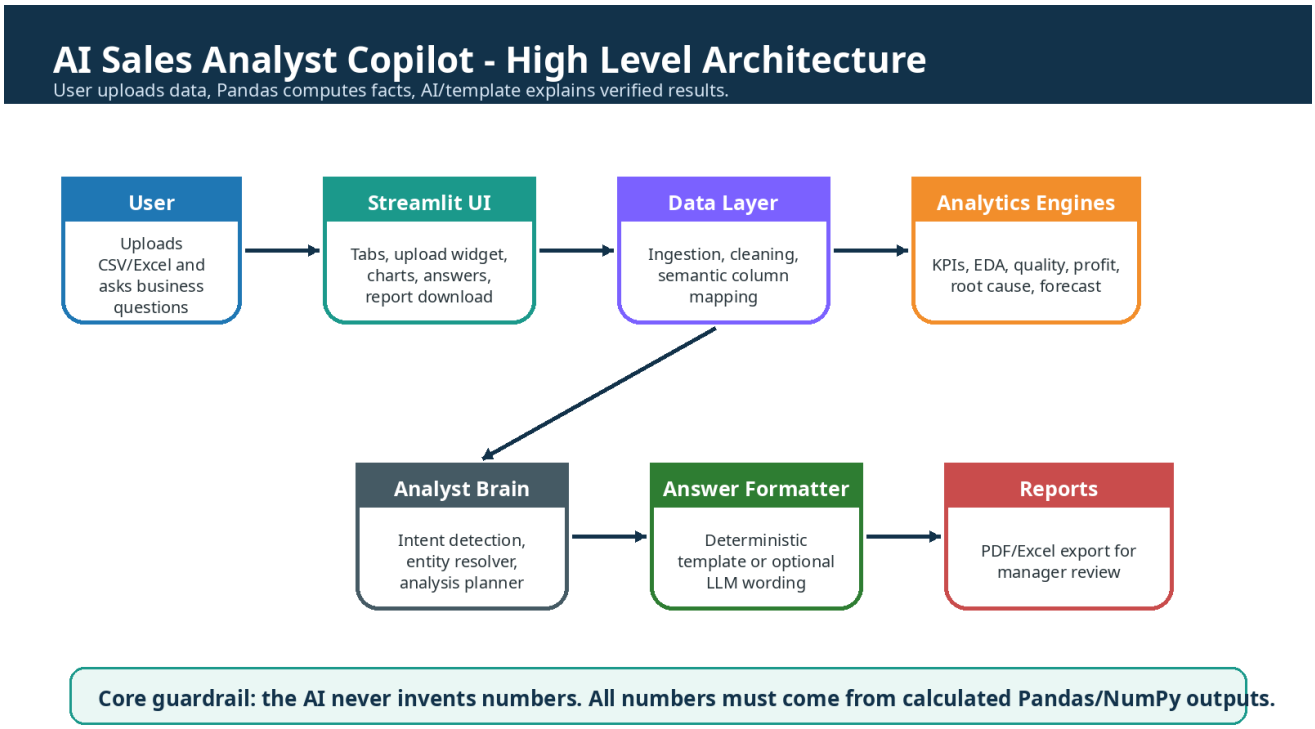
5. Tech Stack

The current MVP is a local Streamlit application. A future production version can split the app into a FastAPI backend and a React frontend, but the present project can run completely from Python and Streamlit.

Technology	Used for	Why it is used
Python 3.12+	Main programming language	Strong ecosystem for analytics, automation, testing, and AI integration.
Streamlit	User interface	Fast way to build an interactive data app without HTML/CSS.
Pandas	DataFrames, cleaning, groupby, aggregations	Core deterministic calculation engine; prevents AI from guessing.
NumPy	Numerical operations	Supports efficient numeric/statistical calculations.
Plotly	Interactive charts	Useful for dashboards, trends, bars, scatter plots, and visual EDA.
Scikit-learn	Forecasting/anomaly/ML helpers	Provides models and preprocessing tools for optional prediction features.
OpenPyXL	Excel file support	Reads and writes Excel workbooks.
ReportLab	PDF report generation	Creates shareable PDF reports from calculated insights.
python-dotenv	Environment variables	Keeps API keys and optional settings outside source code.
Pytest	Testing	Validates modules and known-answer cases.
OpenAI/Ollama optional	Answer wording / local LLM support	Adds natural-language explanation but the app must still work without paid keys.
Git/GitHub	Version control	Tracks changes and makes the project professional for review.

6. System Architecture

The architecture separates user interface, data handling, deterministic analysis, and answer explanation. This separation is what makes the project reliable. The UI does not calculate everything directly; it calls functions from source modules. The answer layer does not invent answers; it receives a calculated evidence package.



Layer-by-layer explanation

Layer	Responsibility
UI Layer	Streamlit tabs, upload widget, data preview, KPI cards, charts, text answers, download buttons.
Ingestion Layer	Reads CSV/Excel, validates file type, handles parse errors, preserves original dataset.
Semantic Layer	Maps messy/alternate column names to business roles such as sales, profit, date, discount, category.
Analytics Layer	Runs Pandas/NumPy calculations for KPIs, EDA, quality checks, profit intelligence, root cause, forecasts.
Analyst Brain	Classifies user intent, resolves entities, plans which tool/function should answer the question.
Answer Layer	Formats evidence into clear text; optionally uses LLM only to polish wording.
Report Layer	Creates PDF/Excel outputs from the same calculated evidence.

7. Folder Structure

A clean project structure makes the app easier to debug, test, explain, and extend. Each module has one job. This is better than putting all logic inside one app.py file.

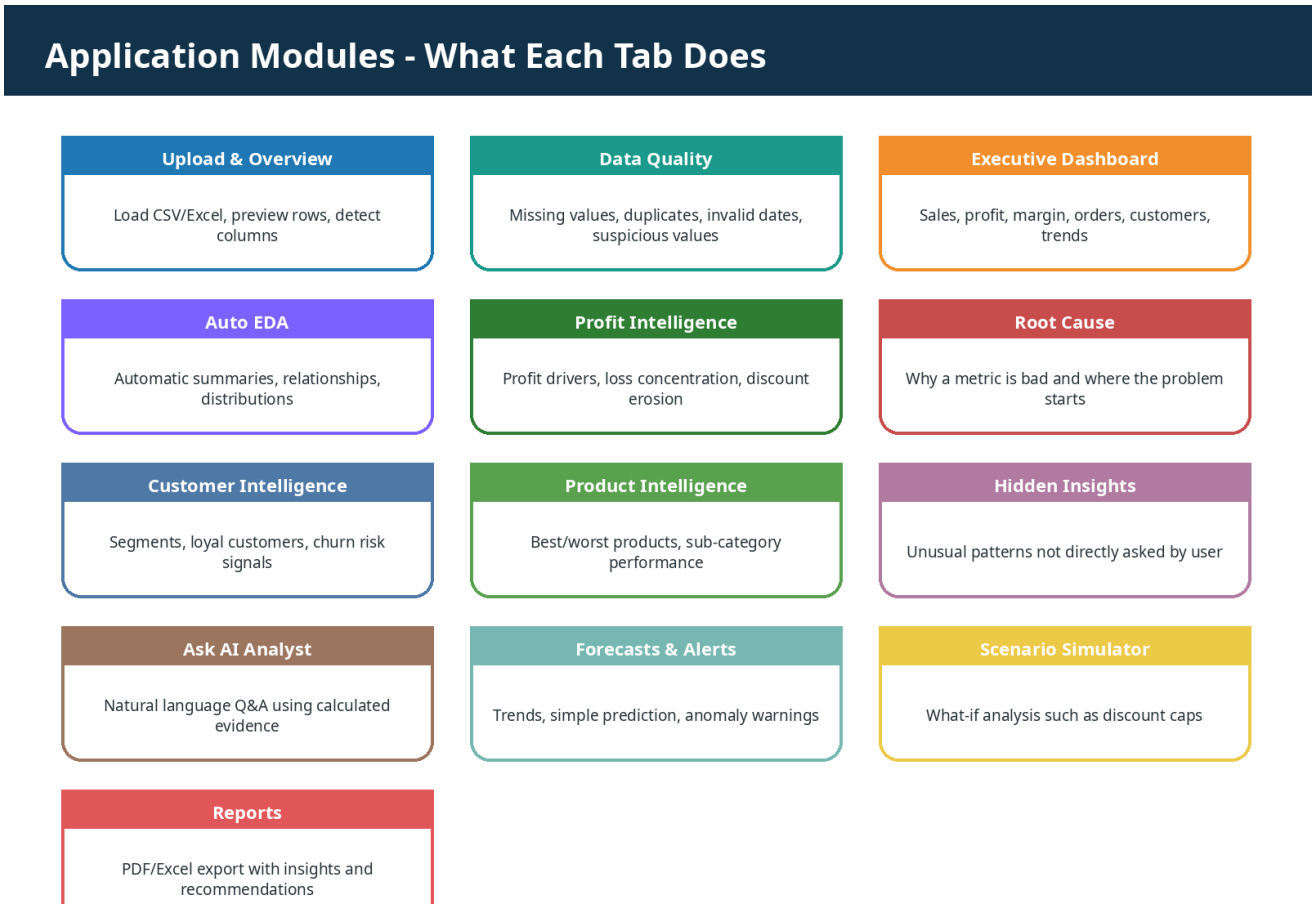
```
ai_sales_analyst_copilot/
|-- app.py
|-- requirements.txt
|-- README.md
|-- .env.example
|-- .gitignore
|-- sample_data/
|   |-- generate_sample_sales_data.py
|-- reports/
|-- src/
|   |-- ingestion.py
|   |-- column_detection.py
|   |-- semantic_layer.py
|   |-- data_quality.py
|   |-- kpi_engine.py
|   |-- eda_engine.py
|   |-- metric_relationships.py
|   |-- profit_intelligence.py
|   |-- root_cause.py
|   |-- customer_intelligence.py
|   |-- product_intelligence.py
|   |-- region_intelligence.py
|   |-- recommendation_engine.py
|   |-- analyst_brain.py
|   |-- analysis_planner.py
|   |-- entity_resolver.py
|   |-- question_answering.py
|   |-- hidden_insights.py
|   |-- forecasting.py
|   |-- anomaly_detection.py
|   |-- scenario_simulator.py
|   |-- report_generator.py
|   |-- utils.py
|-- tests/
    |-- test_ingestion.py
    |-- test_column_detection.py
    |-- test_data_quality.py
    |-- test_kpi_engine.py
    |-- test_question_answering.py
    |-- test_brain_questions.py
```

Beginner explanation of key files

File	Role
app.py	Main Streamlit file. It builds the page layout and calls source modules.
requirements.txt	List of Python packages needed to run the project.
.env.example	Template for optional API/model settings.
src/*.py	Reusable business logic modules. This is where calculations should live.
tests/*.py	Automated checks to prove the app works.
reports/	Generated PDF/Excel reports can be saved here.

8. Full Module Breakdown

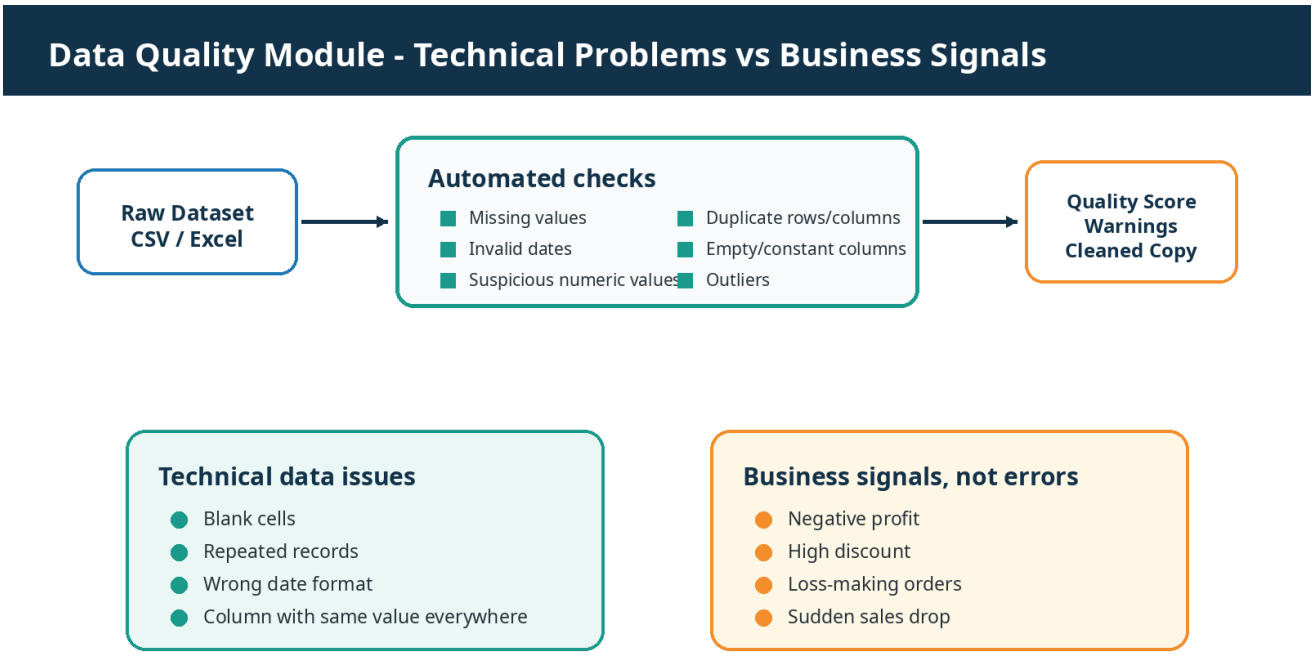
The app is split into tabs/modules. A beginner should understand each module as a separate worker. The upload module brings data in. The quality module checks trust. The KPI and EDA modules summarize performance. The intelligence modules explain where profit/loss/customer/product/region issues are happening. The Ask AI Analyst module makes the app conversational.



Module	Input	Processing	Output
Upload & Overview	CSV/Excel file	Read file, preview DataFrame, list columns/dtypes, detect shape	Dataset preview and basic metadata
Data Quality	Uploaded DataFrame	Check missing, duplicates, invalid dates, empty/constant columns, suspicious values	Quality score, issue list, warnings
Executive Dashboard	Mapped KPI columns	Calculate sales, profit, margin, orders, customers, discount	KPI cards and charts
Auto EDA	Numeric/categorical/date columns	Summaries, distributions, correlations, top/bottom groups	Automatic insights and plots
Profit Intelligence	Sales, profit, cost, discount, product/category columns	Margin analysis, loss orders, discount erosion, loss concentration	Profit drivers and leakage areas
Root Cause Analysis	Target metric/entity	Drill from category to sub-category/product/region/segment/discount	Why the issue is happening
Customer Intelligence	Customer, order, sales, profit columns	Customer revenue, loyalty score, churn/at-risk signals	Best customers and risky customer groups
Product Intelligence	Product/category/sub-category columns	Best/worst products, profit ranking, sales contribution	Product action list
Hidden Insights	Full dataset	Scan for unusual patterns, concentration, anomalies, relationships	Insights user may not ask directly
Ask AI Analyst	Question + semantic mapping	Intent routing, entity resolution, Pandas tool execution, answer formatting	Conversational answer with evidence
Forecasts & Alerts	Date + metric columns	Trend analysis, simple forecast, anomaly detection	Warnings and future trend estimate
Scenario Simulator	User scenario such as discount cap	Change assumption, recalculate metrics, compare before/after	What-if impact
Reports	Calculated results, charts, recommendations	Generate PDF/Excel report	Downloadable manager-ready files

9. Data Quality and Cleaning Logic

Before trusting any business insight, the app checks whether the dataset itself is usable. This is important because bad data leads to bad decisions. The project separates technical data issues from business signals. For example, missing dates are technical issues, but negative profit is not automatically a data error; it may be a real business loss that should be analyzed.



Cleaning principles

Rule	Reason
Preserve original dataset	A user must be able to compare raw data with cleaned data.
Create a separate cleaned copy	Avoid silently changing the source data.
Maintain treatment log	Every fill/drop/derived column should be traceable.
Do not fake dates	Wrong dates damage trend analysis.
Treat negative profit as business signal	Losses are important; they should not be removed as outliers automatically.
Show warning when cleaned data is used	The user should know when results depend on imputed/cleaned values.

10. KPI, EDA, Profit Intelligence, and Root Cause

KPI Engine

The KPI engine calculates headline business metrics. These are the numbers a manager will ask for first. The engine should calculate totals and ratios using detected semantic columns, not fixed hardcoded column names.

KPI	Formula idea	Meaning
Total Sales	sum(sales_column)	How much revenue was generated.
Total Profit	sum(profit_column)	How much profit was made after costs/discounts.
Profit Margin	total_profit / total_sales	How much profit is kept from each rupee/dollar of sales.
Average Order Value	mean(order_sales)	Typical sales value per order/row depending on data granularity.
Loss-Making Orders	count(profit < 0)	How many records are losing money.
Average Discount	mean(discount_column)	How much discounting is happening.

EDA Engine

EDA finds patterns without the user asking a very specific question. It can show numeric summaries, categorical rankings, date trends, correlations, distributions, and top/bottom performers. Auto EDA should avoid crashes on messy data, zero-variance columns, missing values, and non-standard date formats.

Profit Intelligence

Profit intelligence goes deeper than total profit. It answers where profit comes from, where it leaks, how loss is concentrated, whether high discounts damage margin, and which category/product/region/segment needs action.

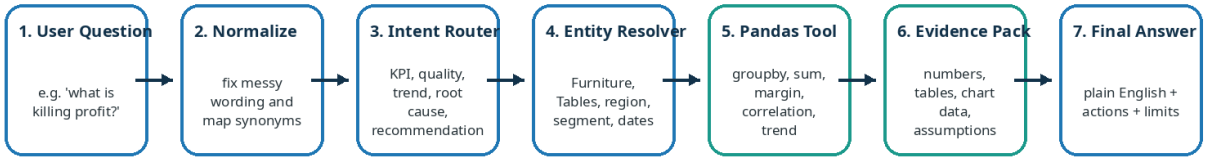
Root Cause Analysis

Root cause analysis starts with a problem and drills down until it finds the strongest evidence. For example, if Furniture is low-profit, the app checks sub-categories, products, discounts, regions, customer segments, and time trends. The final answer should say what is causing the issue, not just repeat the total loss.

11. Ask AI Analyst and Tier 4 Messy Question Handling

The hardest part of this project is natural-language question answering. Real users do not always ask clean questions. They may use wrong terms, incomplete language, emotional phrasing, or follow-up questions. The app needs an interpretation layer before running calculations.

Ask AI Analyst - Safe Question Answering Flow



Guardrails that make the answer trustworthy

- **No guessing:** If the data does not contain a required column, the answer says what is missing.
- **No unsafe eval:** The user question is never executed as raw Python code.
- **No hardcoded dataset:** Semantic roles allow alternate names like Revenue = Sales.
- **API key optional:** Without OpenAI/Ollama, the app uses deterministic template answers.
- **Tier 4 support:** Messy questions are interpreted before running calculations.

Question tiers

Tier	Question type	Example	Expected handling
Tier 1	Direct dataset facts	How many rows and columns?	Return exact profile value.
Tier 2	Business metric questions	Which category is most profitable?	Group by category and rank profit.
Tier 3	Reasoning with clear intent	Is discount affecting profit?	Calculate relationship, compare discount bands, explain evidence.
Tier 4	Messy real-world questions	What is killing our profit?	Interpret intent, identify profit root-cause path, calculate drivers, answer with limits.

Tier 4 interpretation examples

User wording	What it probably means	Module/tool to use
non unique	Duplicate rows or duplicate values	Data quality / duplicate check
how are we doing?	Overall KPI summary	Executive dashboard + KPI engine
what is killing our profit?	Profit root-cause analysis	Profit intelligence + root cause
tell me about discounts	Discount distribution and impact on profit	Metric relationships + profit intelligence
which is better?	Ambiguous comparison	Ask clarification unless context exists
Furniture is profitable right?	Verify assumption	Category profit/margin calculation
give sales and tell me what to fix	Multi-intent: KPI + recommendation	KPI engine + recommendation engine

12. Reporting, Export, and Manager Review Output

The report module converts analysis into a shareable deliverable. This is important because managers usually do not want raw code or terminal output. They want a clean PDF or Excel file containing the problem, data summary, KPIs, charts, findings, recommendations, and limitations.

Report section	Content
Dataset summary	Rows, columns, date range, detected business columns, missing values, duplicates.
Executive KPIs	Sales, profit, margin, orders, customers, loss-making orders, discount.
Charts	Category profit, sales trend, discount vs profit, top/bottom products, region/segment view.
Root-cause analysis	Evidence-based explanation of the strongest loss/profit drivers.
Recommendations	Actions such as discount guardrails, product review, pricing review, region focus.
Limitations	Missing columns, cleaned values, assumptions, absence of cost/returns/ship-cost data.

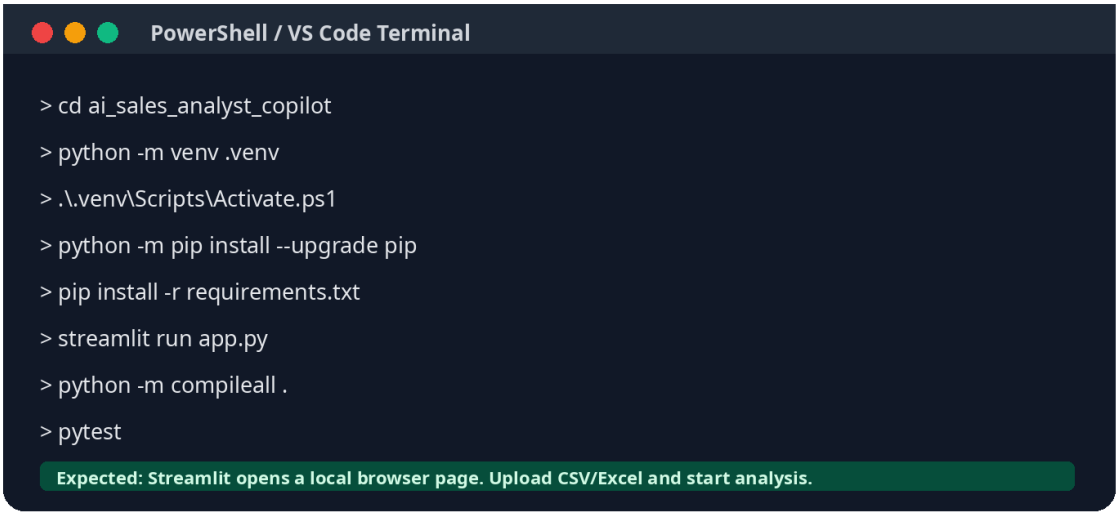
Recommended report rule

The report should use the same calculated evidence as the UI. It should not regenerate numbers independently through the LLM. That keeps the PDF/Excel output consistent with the dashboard and Ask AI answers.

13. How to Run the Project

Use these steps on Windows in VS Code or PowerShell. The project should run without a paid API key. Optional OpenAI/Ollama settings can be added later in the .env file.

How to Run the Project Locally



Detailed Windows setup

```
# 1. Open VS Code terminal inside the project folder
cd ai_sales_analyst_copilot

# 2. Create virtual environment
python -m venv .venv

# 3. Activate virtual environment
.\.venv\Scripts\Activate.ps1

# If activation is blocked, run this once in PowerShell as your user:
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

# 4. Install dependencies
python -m pip install --upgrade pip
pip install -r requirements.txt

# 5. Run the Streamlit app
streamlit run app.py

# 6. Test code health
python -m compileall .
pytest
```

If you do not want to activate the environment

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m streamlit run app.py
```

Optional .env setup

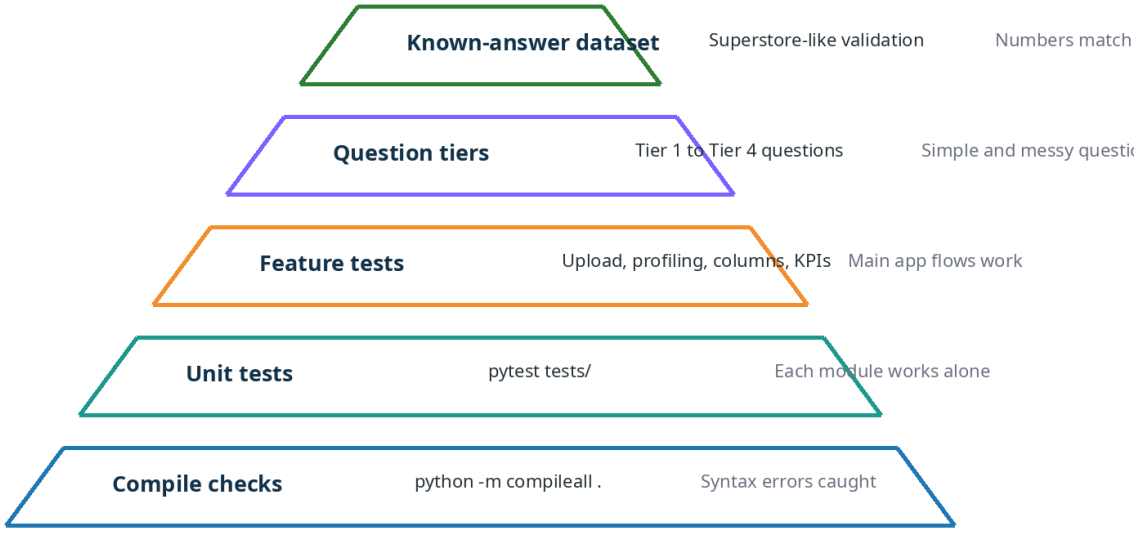
```
# .env
OPENAI_API_KEY=
OLLAMA_API_URL=http://localhost:11434
```

Leave API values blank if you do not want to use any external or local LLM. The app should still answer using deterministic templates.

14. Testing and Validation Plan

Testing is not only checking whether the app opens. The project must prove that each feature returns correct values, handles bad input, and does not break when the dataset changes.

Testing Strategy - How We Know the App Works



Rule: a feature is complete only when it gives the right answer, handles bad inputs, and does not crash.

Core test checklist

Area	Must verify
Upload	CSV loads, Excel loads, invalid file shows clear error, question before upload does not crash.
Profiling	Correct row/column counts, dtypes, numeric/categorical/date detection, missing/duplicate counts.
Column detection	Sales/profit/discount/date/customer/category/product/region detected even with alternate names.
Manual override	User can change detected columns and all answers update immediately.
Data quality	Missing, duplicates, invalid dates, empty/constant columns, suspicious values; business losses are separate from quality errors.
KPI engine	Sales, profit, margin, orders, customers, AOV, discounts, loss orders calculated correctly.
EDA	Charts and summaries handle numeric/categorical/date columns without zero-variance or missing-value crashes.
Q&A;	Tier 1 to Tier 4 questions return calculated evidence, not hallucinated numbers.
Reports	PDF/Excel downloads are created and match dashboard numbers.

Known-answer sample validation

For a Superstore-like validation dataset, you can use known answers to test correctness. These values are sample-specific, not universal: 9994 rows, 21 columns, total sales around $2,297,200.86, total profit around $286,397.02, 793 unique customers, 5009 unique orders, and Tables as a major loss-making sub-category around -$17,725.48.

15. Build-From-Scratch Roadmap

A beginner should not build everything at once. Build the app in layers. Each layer must work before moving to the next one.

Stage	Build	Checkpoint
1	Create project folder, virtual environment, requirements.txt, app.py	Streamlit opens successfully.
2	File upload and DataFrame preview	CSV/Excel upload works and invalid files show errors.
3	Dataset profiling	Rows, columns, dtypes, missing, duplicates appear correctly.
4	Column detection and semantic layer	Sales/profit/date/category detected across alternate names.
5	KPI engine	Total sales/profit/margin/order/customer numbers match known dataset.
6	Data quality module	Technical issues and business signals separated.
7	EDA and charts	Top/bottom, correlations, date trends, category summaries render correctly.
8	Profit intelligence	Loss areas, discount erosion, margin drivers calculated.
9	Root cause module	Question like 'why is Furniture losing money?' returns evidence.
10	Ask AI Analyst	Intent router + entity resolver + Pandas tool execution + answer formatter.
11	Forecasting, anomaly, simulator	Trends, warnings, what-if scenarios work with required columns.
12	Reports and polish	PDF/Excel export, README, tests, screenshots, GitHub-ready repo.

16. YouTube Resources to Build This Project

Use these resources in order. Do not only watch. After each resource, implement the matching project part immediately.

Python basics - Learn Python - Full Course for Beginners [Tutorial]

Build after watching: Implement simple functions and read/write files.
https://www.youtube.com/watch?v=rfscVS0vtbw

Pandas + data analysis - Data Analysis with Python for Excel Users - Full Course

Build after watching: Upload CSV and calculate profile/KPIs.
https://www.youtube.com/watch?v=WcDaZ67TVRo

Streamlit UI - Build a Streamlit Dashboard app in Python

Build after watching: Create tabs, upload widget, dashboard layout.
https://www.youtube.com/watch?v=p2pXpcXPoGk

Streamlit crash course - Streamlit Crash Course: From Zero to Data App

Build after watching: Understand Streamlit basics quickly.
https://www.youtube.com/watch?v=d7fnzDQ5qM8

FastAPI future backend - FastAPI Course for Beginners

Build after watching: Optional future API backend.
https://www.youtube.com/watch?v=tLKKmouUams

Plotly charts - Learn Python Plotly Data Visualization with 10 Practical Examples

Build after watching: Create dashboard charts.
https://www.youtube.com/watch?v=YMa-l_0ZHmw

Scikit-learn - Machine Learning with Python and Scikit-Learn - Full Course

Build after watching: Forecasting/anomaly detection basics.
https://www.youtube.com/watch?v=hDKCxebp88A

Git/GitHub - Git and GitHub for Beginners - Crash Course

Build after watching: Push project to GitHub professionally.
https://www.youtube.com/watch?v=RGOj5yH7evk

Docker - Docker Tutorial for Beginners - Full DevOps Course

Build after watching: Package the app later for reproducible deployment.
https://www.youtube.com/watch?v=fqMOX6JJhGo

ReportLab PDF - How to Create a PDF Using Python

Build after watching: Build PDF report export.
https://www.youtube.com/watch?v=b4QxB5c5kzc

RAG future module - RAG From Scratch

Build after watching: Future PDF/document Q&A; with citations.
https://www.youtube.com/watch?v=wd7TZ4w1mSw

LangGraph future agent - LangGraph Complete Course for Beginners

Build after watching: Future graph-based analyst workflow.
https://www.youtube.com/watch?v=jGg_1h0qzaM

Function/tool calling - OpenAI Function Calling For Beginners

Build after watching: Future tool-based AI question routing.
https://www.youtube.com/watch?v=78x1dB4poaE

Ollama local LLM - Run LLMs Locally - Ollama Beginner Tutorial

Build after watching: Optional local model support without paid API.
https://www.youtube.com/watch?v=KGY0_0j8MZQ

SQL analytics - SQL for Data Analytics - Learn SQL in 4 Hours

Build after watching: Future database-backed analytics.
https://www.youtube.com/watch?v=7mz73uXD9DA

17. Limitations and Future Scope

Limitation	What the app should do
Dataset missing profit/cost columns	Say true profitability cannot be calculated; use sales-only or estimate only if valid columns exist.
Messy column names	Use semantic detection and manual override.
Large datasets	Add caching, chunking, database storage, or optimized queries.
LLM hallucination risk	Keep all numeric answers grounded in Pandas evidence.
Ambiguous questions	Ask a short clarification if there is not enough context.
Forecast limitations	Use simple forecasts carefully and label them as estimates.

Future upgrades

- FastAPI backend and React frontend for production deployment.
- Database support using PostgreSQL or SQLite for larger datasets and history.
- RAG module for uploaded PDFs/business documents.
- LangGraph workflow for multi-step agentic analysis.
- LangSmith or custom evaluation logs to test answer quality.
- Role-based dashboards for manager, analyst, and executive users.
- Better forecasting and anomaly models with clear confidence/limitations.
- One-click manager report with charts and evidence appendix.

18. How to Explain This Project in an Interview or Manager Review

30-second explanation

I built an AI Sales Analyst Copilot that lets a user upload a CSV/Excel sales dataset and automatically generates data profiling, data quality checks, KPIs, EDA, profit intelligence, root-cause analysis, natural-language Q&A, and PDF/Excel reports. The key design choice is that all numeric answers are calculated with Pandas and NumPy, while AI is only used to explain verified results in business language.

Technical explanation

The app uses Streamlit for the interface and modular Python files for ingestion, semantic column detection, data quality, KPIs, EDA, profit intelligence, root cause, recommendation generation, and question answering. The question answering layer routes messy business questions to deterministic tools, executes calculations on the uploaded DataFrame, creates an evidence pack, and formats a final answer with assumptions and limitations.

Resume bullets

- Built a Streamlit-based AI Sales Analyst Copilot that analyzes CSV/Excel sales datasets and generates business insights without requiring SQL or Python from the end user.

- Implemented deterministic Pandas/NumPy engines for data profiling, KPI calculation, data quality checks, EDA, profit intelligence, and root-cause analysis.

- Designed a semantic column detection layer to map alternate column names such as Revenue, Total Amount, Sales, Margin, and Sub-Category to business roles.

- Created a guarded natural-language Q&A pipeline where the AI explains calculated evidence instead of hallucinating numbers.

- Added testing strategy for upload handling, profiling, column detection, KPIs, data quality, Tier 1-Tier 4 questions, and report export.

Final project rule

The project is successful only when a beginner can upload data, understand the results, ask business questions, and receive trustworthy answers backed by calculated evidence.